



Framsticks

 Search


Documentation

Downloads

Creatures

Experiments

Development

Forum

- ▶ [Documentation](#)
- ▶ [Graphics and videos](#)
- ▶ [Downloads](#)
- [Creatures](#)
- ▶ [Experiments](#)
- ▶ [Development](#)
- ▶ [Community](#)

On the forum:

- [Framsticks 5.2 released!](#)
- [Online Framsticks Creature Editor updated](#)
- [Framsticks 5.0 released!](#)
- [Evolutionary experiments and exercises in Python](#)
- [Framsticks mobile versions available](#)

[More](#)

Genotype (format f0 and f0s)

"Format Zero" genotype is the construction plan for any Framsticks creature. More detailed information on how to use it for development of one's own stick objects is available in the [Framsticks SDK](#).

f0 and **f0s** share exactly the same syntax. The difference is the type of shapes used for Parts and Joints. **f0** is for ball-and-stick shapes, while **f0s** is for solid shapes (ellipsoids, cuboids, cylinders, hinges).

f0/f0s Syntax

Each line in **f0** or **f0s** describes one object in the model.

Lines starting with the # sign are considered to be comments and are ignored.

The syntax is:

```
CLASSID:PROPERTY1,PROPERTY2,...
```

CLASSID – alphanumeric identifier of the class. Currently, there are four object classes (see [f0/f0s Semantics](#))

PROPERTY1, PROPERTY2,... – a set of properties of the object. Each class defines the **sequence** of properties and a reasonable **default value** for each one. The full definition of the property is NAME=VALUE. You can skip NAME= if the given property is next in the sequence. Object descriptions with omitted property names are not recommended as they may be misinterpreted by future SDK versions with different property sets. There is an exception for easier editing: skipping inside the "natural" (= "unlikely to change") property sequences like x,y,z is allowed, you can write "x=1,2,3" instead of "x=1,y=2,z=3". (Such exceptional properties are marked by flag 1024 in the property definition, see Param::flags(int i)). You can also skip the whole definition of a property and accept the default value. If you want to pass special characters (for ex. a comma) or set the empty string value (""), VALUE can be placed between quotes ("").

Examples

Let's assume the following class definition: CLASSID=ob and four properties with default values: a=0, b=1, c=2, d=3

Examples of valid **f0/f0s** descriptions:

all properties are set explicitly

ob: a=9, b=8, c=7, d=6

you can omit property names if the sequence is obeyed	ob:9,8,7,6
some properties have no names, but they are deduced from the sequence	ob:d=6,a=9,8,7
"b" and "c" are not defined – default values will be used (b=1,c=2)	ob:9,, ,7
b="", c=2	ob:9,"" , ,7
b="," (comma), c=2	ob:9,"" , ,7
default values used for "a","c" and "d"	ob:b=8
default values used for all properties	ob:

Note that values can be surrounded by quotes ("). So `ob:c="x-y"` and `ob:c=x-y` are equivalent. Inside quotes, quoting takes place: `ob:d="x=\"y\""`. For details, see `param.cpp` (in SDK), function `ParamInterface::load2`.

f0/f0s Semantics

Parts, Joint and Neurons have **reference numbers** used to attach other objects. References start with 0 and every new object in the class gets the next reference number.

Part object

Creates instance of the Part object.

- **CLASSID=p:**
- **x=0, y=0, z=0** – position in 3D
- **rx=0, ry=0, rz=0** – 3D orientation (amount of rotation around each of the 3 axes)
- **sh=0** – Part shape:
 - 0=ball (for ball-and-stick models and **f0**),
 - 1=ellipsoid, 2=cuboid (box), 3=cylinder (for solid models and **f0s**).

The default shape=0 builds the classic Framsticks structure where Parts are "balls" and Joints are "sticks". Shapes above 0 allow for detailed customization of creature elements. These shapes can be scaled independently in 3 axes using `sx,sy,sz` properties. Each scaling factor is interpreted as radius, for example `p:sh=1,sx=0.5,sy=0.5,sz=0.5` is the spherical Part (special case of ellipsoid) of radius 0.5 (and diameter 1.0). Parts having shapes above 0 ("solid type" Parts) must not be mixed with "ball-and-stick type" model elements (having shape=0).

- **s=1** – size as radius (for collisions only, for "ball-and-stick type" Parts having shape=0)
- **sx=1, sy=1, sz=1** – shape size as radius ("solid type" Parts having shape>0)

- **dn=1** – density. Influences buoyancy under water in the MachaStick simulator. Note that the mass of a Part is calculated and cannot be changed. For **f0**, the mass of each Part is calculated as the number of its incident Joints. For **f0s**, the mass of each Part is calculated internally as its density*volume.
- **fr=0.4** – friction
- **ing=0.25** – ingestion
- **as=0.25** – assimilation
- **vr=0.5, vg=0.5, vb=0.5** – color (RGB)
- **i=""** – general purpose "info" field

The Part's position and orientation can be overridden by a Joint definition using its *delta option*. In this case, you can skip x, y and z in the Part definition (see [f0 Examples](#)).

Joint object

Creates instance of the Joint object.

- **CLASSID=j**:
- **p1=-1, p2=-1** – ref.numbers of the two connected Parts. Note that the default -1 is illegal here, you must not omit these properties. Both Parts have to be created earlier.
- **rx=0, ry=0, rz=0** – relative rotation only when *delta option* (see below) has been activated, otherwise ignored. Rotation can be used to enforce specific orientation of the second Part (p2) relative to the first Part (p1). Orientation can influence some of the effectors/receptors and can be used as a growing direction while creature is being constructed.
- **dx, dy, dz** – delta option: if none of these is specified, then the *delta option* is inactive. If any of these is specified, then the remaining ones default to zero, and all three deltas are used as a displacement applied to the second Part (p2) relative to the first Part (p1). Local coordinate system of p1 is first rotated by [rx,ry,rz] and then translated by [dx,dy,dz], and the result constitutes the location and orientation of p2. This technique allows you to define relative placement of Parts. With *delta option* inactive, absolute Part positioning and absolute orientation is used, as defined by [x,y,z] and [rx,ry,rz] of the Part p2.
- **sh=0** – Joint shape:
 - 0=stick (for ball-and-stick models and **f0**),
 - 1=solid connection, 2=single-axis hinge, 3=dual-axis hinge (for solid models and **f0s** – see also hx, hrx... fields below).

Parts having shapes above 0 ("solid type" Parts) must be connected with "solid type" Joints. 1=solid connection means that interconnected Parts are glued into a single solid body. Joint shapes 2 and 3 are articulated joints that can be controlled by the 'M' muscle neuron.

- **stif=1** – stiffness
- **rotstif=1** – rotation stiffness
- **stam=0.25** – stamina
- **vr=0.5, vg=0.5, vb=0.5** – color (RGB)
- **i=""** – general purpose "info" field

The following 10 fields are only valid for **f0s**:

- **hx=0, hy=0, hz=0** – hinge location (relative to Part p1)
- **hrx=0, hry=0, hrz=0** – hinge orientation (relative to Part p1)
- **hxn=- $\pi/2$, hxp=+ $\pi/2$** – x-axis hinge rotation range (for single- and dual-axis hinges)
- **hyn=- $\pi/2$, hyp=+ $\pi/2$** – y-axis hinge rotation range (for dual-axis hinges)

For illustrations of relative translation and rotation, see the bottom of [this page](#).

The Joint objects do not exist as material objects in simulation. However, for "ball-and-stick type" constructs with shapes=0, Joints can be visualized as cylinders with appropriate length and radius=0.2.

Neuron object

Creates instance of the Neuro object.

A signal processing unit, sensor, or effector. Neuron reference number is used in f0docneuroitem definition.

- **CLASSID=n:**
- **p=-1** – ref.number of the Part the neuron is attached to (the part has to be created earlier).
- **j=-1** – ref.number of the Joint the neuron is attached to (the joint has to be created earlier).
- **d** – neuron class description, like "classname:properties_and_values"
- **i=""** – general purpose "info" field

Neuron input definition (connection)

Add input to the neuron = weighted connection to the other neuron. Both objects must be already created.

- **CLASSID=c:**
- **n** – ref.number of the parent neuron
- **i** – ref.number of the neuron to be connected as the parent's input
- **w=1.0** – connection weight (optional)

Model properties (optional)

- **CLASSID=m:**
- **se=1.0** – starting energy (relative to default)
- **Vstyle=""** – the name of the visual style. This field can be used to change the appearance of the creature. Several visual styles are predefined: **food**, **mark**, **boid**, **foram**, and **manipulator**. For **food** and **mark**, the additional, optional RGB color argument can be specified (for example `Vstyle="food(color=0x00ff00)"`). Custom visual styles can be defined for wireframe and OpenGL views – see WireframeAppearance and Visual style definition in the [script class documentation](#).

Model validity constraints

- at most one Joint can directly link two Parts
- each Joint must be connected with two distinct Parts
- all Parts must be directly or indirectly connected with each other
- delta Joints must not form cycles
- for each Joint, the Part-to-Part distance must not exceed the value defined as the maximum Joint.dx (default=2.0)

f0 Examples

To see these in Framsticks application, you have to start writing **f0** genotype with `//0` (two slashes and zero and new line). The shortest **f0** genotype is a single Part:

p:

A single stick, "X" in **f1**

p:

p:1

j:0,1

Three sticks in line, "XXX" in **f1**, no "delta option" – absolute coordinates used in all Parts

p:

p:1,m=2

p:2,m=2

p:3,

j:0,1

j:1,2

j:2,3

Three sticks line, "XXX" in **f1**, with "delta option" – relative positioning (dx=1)

p:

p:m=2

p:m=2

p:

j:0,1,dx=1

j:1,2,dx=1

j:2,3,dx=1

Three sticks star (120 degrees), "X(X,X)" in **f1**, no delta option, absolute coordinates are awkward and the sticks' length cannot be seen

p:

p:1, m=3

p:1.50017, -0.865927

p:1.50017, 0.865927

j:0, 1

j:1, 2

j:1, 3

Three sticks star (120 degrees), "X(X,X)" in **f1**, using delta option, dx=1, note that $120 \text{ degrees} / 2 = 1.047 \text{ rad}$

p:

p:m=3

```

p:
p:
j:0, 1, dx=1
j:1, 2, rz=-1.047, dx=1
j:1, 3, rz=1.047, dx=1

```

Curved chain of 3 sticks, similar to CXXX. Note that each **rz** in **j**: not only affects direction of a joint, but also changes the orientation of the second Part, which propagates to all subsequent Parts through subsequent **j**: **rz**

```

p:
p:
p:
p:
j:0,1,dx=1
j:1,2,dx=1,rz=0.5
j:2,3,dx=1,rz=0.5

```

Neuron net example, "X[G:1,1:2.3][@-1:3.4,0:4.5,T:5.6]" in **f1**

```

p:
p:1
j:0, 1, dx=1
n:p=1
n:j=0, d="| :p=0.25,r=1"
n:j=0, d=G
n:p=1
n:j=0, d=@:p=0.25
n:p=1, d=T
c:0, 2
c:0, 3, 2.3
c:1, 0
c:3, 0, 3.4
c:3, 3, 4.5
c:3, 5, 5.6
c:4, 3

```

Cyclic structure, Parts are connected 0->1->2->3->0. Not possible in **f1** or **f4** formats.

```

p:0,0
p:1,0
p:1,1
p:0,1
j:0,1
j:1,2
j:2,3
j:3,0

```